NAME = ARAYANA KAHRE
SAP ID = 590042432
BATCH= 15

1. Aim

To write a menu-driven C program using separate user-defined functions to perform the following operations:

1. Remove all vowels from a string.

2. Find the first repeating lowercase alphabet in a string.

3. Check whether two strings are anagrams of each other.

2. Algorithm

1. Start the program.

2. Read a string or two strings according to the selected operation.

3. Display the menu with three operations and an Exit option.

4. If the user selects option 1, remove all vowels from the string and display the modified string.

5. If the user selects option 2, find the first lowercase alphabet that repeats and display it.

6. If the user selects option 3, check whether both strings contain the same letters with the same frequencies.

7. Display the result.

8. Repeat the menu until the user selects Exit.

9. Stop the program.

3. Complete C Program

यह program आसान भाषा में लिखा गया है और इसमें हर operation के लिए अलग user-defined function है।

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define SIZE 200

void removeVowels(char str[]);
char findFirstRepeating(char str[]);
int checkAnagram(char str1[], char str2[]);

int main()
{
    char str1[SIZE], str2[SIZE];
    int choice;

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Remove all vowels\n");
        printf("2. Find first repeating lowercase
alphabet\n");
        printf("3. Check anagrams\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        getchar();

        switch (choice)
        {
            case 1:
                printf("Enter a string: ");
                fgets(str1, SIZE, stdin);
                str1[strcspn(str1, "\n")] = '\0';
```

```
                removeVowels(str1);
                printf("String after removing vowels: %s\n",
str1);
                break;

            case 2:
            {
                char result;

                printf("Enter a string: ");
                fgets(str1, SIZE, stdin);
                str1[strcspn(str1, "\n")] = '\0';

                result = findFirstRepeating(str1);

                if (result == '\0')
                    printf("No repeating lowercase alphabet
found.\n");
                else
                    printf("First repeating lowercase
alphabet: %c\n",
                           result);
                break;
            }

            case 3:
            {
                int result;

                printf("Enter first string: ");
                fgets(str1, SIZE, stdin);
                str1[strcspn(str1, "\n")] = '\0';

                printf("Enter second string: ");
                fgets(str2, SIZE, stdin);
                str2[strcspn(str2, "\n")] = '\0';

                result = checkAnagram(str1, str2);

                if (result)
                    printf("The strings are anagrams.\n");
                else
                    printf("The strings are not anagrams.
\n");
```

```
                break;
            }

            case 4:
                printf("Program exited.\n");
                break;

            default:
                printf("Invalid choice!\n");
        }

    } while (choice != 4);

    return 0;
}

void removeVowels(char str[])
{
    int i, j = 0;
    char ch;

    for (i = 0; str[i] != '\0'; i++)
    {
        ch = tolower((unsigned char)str[i]);

        if (ch != 'a' && ch != 'e' && ch != 'i' &&
            ch != 'o' && ch != 'u')
        {
            str[j++] = str[i];
        }
    }

    str[j] = '\0';
}

char findFirstRepeating(char str[])
{
    int seen[26] = {0};
    int i;
    char ch;

    for (i = 0; str[i] != '\0'; i++)
    {
        ch = str[i];
```

```
        if (ch >= 'a' && ch <= 'z')
        {
            if (seen[ch - 'a'] == 1)
                return ch;

            seen[ch - 'a'] = 1;
        }
    }

    return '\0';
}

int checkAnagram(char str1[], char str2[])
{
    int count[26] = {0};
    int i;
    char ch;

    for (i = 0; str1[i] != '\0'; i++)
    {
        ch = tolower((unsigned char)str1[i]);

        if (ch >= 'a' && ch <= 'z')
            count[ch - 'a']++;
    }

    for (i = 0; str2[i] != '\0'; i++)
    {
        ch = tolower((unsigned char)str2[i]);

        if (ch >= 'a' && ch <= 'z')
            count[ch - 'a']--;
    }

    for (i = 0; i < 26; i++)
    {
        if (count[i] != 0)
            return 0;
    }

    return 1;
}
```

4. Step-by-step explanation

Operation 1: Remove all vowels

Function used: `removeVowels(char str[])`

Vowels are `a, e, i, o, u`. The program removes both uppercase and lowercase vowels.

Example:

Input:

```
Enter a string: Hello World
```

Output:

```
String after removing vowels: Hll Wrld
```

Working of the function:

1. `i` scans every character in the original string.

2. `j` stores the position where the next non-vowel character will be placed.

3. `tolower()` converts a character temporarily to lowercase for checking.

4. If the character is not a vowel, it is copied to position `j`.

5. Finally, `str[j] = '\0'` terminates the modified string.

Important code:

```
if (ch != 'a' && ch != 'e' && ch != 'i' &&
    ch != 'o' && ch != 'u')
{
    str[j++] = str[i];
}
```

Operation 2: Find the first repeating lowercase alphabet

Function used: `findFirstRepeating(char str[])`

Example:

Input:

```
Enter a string: abcdbea
```
Output:

```
First repeating lowercase alphabet: b
```
Working of the function:

1. An array `seen[26]` is initialized with zeros.

2. The program scans the string from left to right.

3. It checks only lowercase letters from `a` to `z`.

4. `ch - 'a'` calculates the index of the letter in the array.

5. If that index already contains `1`, the letter is repeating and is returned.

6. Otherwise, the program marks that letter as seen.

7. If no letter repeats, the function returns `'\0'`.

For example, in `abcdbea`, the second occurrence of `b` is encountered before the second occurrence of `a`, so the answer is `b`.

Operation 3: Check whether two strings are anagrams

Function used: `checkAnagram(char str1[], char str2[])`

Two strings are anagrams if they contain the same letters with the same frequencies, regardless of their order.

Example:

Input:

```
Enter first string: listen
Enter second string: silent
```
Output:

```
The strings are anagrams.
```
Working of the function:

1. Create an integer array `count[26]` initialized to zero.

2. Scan the first string and increase the count of each letter.

3. Scan the second string and decrease the count of each letter.

4. Ignore spaces, punctuation and other non-alphabetic characters.

5. Check all 26 array positions.

6. If every count is zero, the strings are anagrams; otherwise, they are not.

For example, `listen` and `silent` contain the same letters, so every final count becomes zero.

5. Sample output for all three operations

Test Case 1 — Remove vowels

Input

```
Enter your choice: 1
Enter a string: Education
```
Output

```
String after removing vowels: dctn
```

Test Case 2 — Find repeating alphabet

Input

```
Enter your choice: 2
Enter a string: programming
```
Output

```
First repeating lowercase alphabet: r
```

Test Case 3 — Check anagrams

Input

```
Enter your choice: 3
Enter first string: listen
Enter second string: silent
```

Output

```
The strings are anagrams.
```

Test Case 4 — Not anagrams

```
Enter your choice: 3
Enter first string: hello
Enter second string: world
The strings are not anagrams.
```

6. Important functions and concepts

Function / concept	Purpose
`fgets()`	Reads a string containing spaces
`strcspn()`	Removes the newline character
`tolower()`	Converts uppercase letters for comparison
`removeVowels()`	Removes vowels from a string
`findFirstRepeating()`	Finds the first repeated lowercase letter
`checkAnagram()`	Checks whether two strings are anagrams
`seen[26]`	Tracks letters already encountered
`count[26]`	Tracks the frequency difference between two strings

7. Result

The menu-driven C program successfully removes vowels from a string, finds the first repeating lowercase alphabet, and checks whether two strings are anagrams using separate user-defined functions.